

データ準備ガイド — API ファースト戦略とクラウドストレージ

基本方針

本ゼミでは可能な限り **API** 経由でデータにアクセスし、ローカルストレージを圧迫しない運用を推奨する。

原則	内容
API ファースト	GEE Earth Engine API / JAXA Earth API で取得できるデータはダウンロードしない
エクスポートは最小限	オフラインフォールバック用など本当に必要なデータだけを Google Drive に書き出す
クラウド保存	どうしてもファイルが必要な場合は Google Drive (GEEから直接エクスポート可能)を第一選択とする

データアクセス早見表

データ	推奨アクセス方法	ストレージ消費	備考
Sentinel-2 NDVI 時系列	GEE API (COPERNICUS/S2_SR_HARMONIZED)	ゼロ	合宿中もオンラインならその場で取得可能
Sentinel-1 後方散乱	GEE API (COPERNICUS/S1_GRD)	ゼロ	同上
ALOS AW3D30 DSM	JAXA Earth API / GEE API	ゼロ	同上
ALOS-2 PALSAR-2 FNF	JAXA Earth API	ゼロ	森林/非森林マップ

データ	推奨アクセス方法	ストレージ消費	備考
GCOM-C SGLI NDVI	JAXA Earth API	ゼロ	光学補完用
GSMaP 降水量	JAXA Earth API	ゼロ	気象補足情報用
MODIS NDVI	JAXA Earth API	ゼロ	長期時系列用
農地ポリゴン(GeoJSON/Shapefile)	農水省公開データ → Google Drive	数MB	一度入手すれば使い回し可能
e-Stat 農業統計	ブラウザ閲覧のみ	ゼロ	ダウンロード不要
オフライン用 GeoTIFF (S-2/S-1)	GEE エクスポート → Google Drive	数十～数百MB	任意・通信断に備える場合のみ
解析結果の出力(NDVI時系列CSV等)	GEE エクスポート → Google Drive	数十MB	発表資料作成時に参照

Google Drive のセットアップ

Colab から Google Drive をマウントする

Colab から Google Drive 上のファイルを読み書きするための基本手順。

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

実行すると認証リンクが表示されるので、Google アカウントでログインして認証コードをコピー → Colab に貼り付ける。

マウント後は `/content/drive/MyDrive/` 以下に通常のファイル操作でアクセスできる。

```
import os
```

```
os.chdir('/content/drive/MyDrive/astrocamp-2026')
```

```
!ls
```

GEE から Google Drive にエクスポートする

NDVI や SAR 後方散乱の画像を GeoTIFF として Google Drive に保存する。

```
import ee

task = ee.batch.Export.image.toDrive(

    image=ndvi,

    description='ndvi_fukushima_202406',

    folder='astrocamp-2026',

    fileNamePrefix='ndvi_202406',

    region=roi,

    scale=10,

    crs='EPSG:32654',

    maxPixels=1e13

)

task.start()

print(f'Export started: {task.id}')

# 完了状態を確認

import time

while task.active():

    print(f'Status: {task.status()["state"]}')

    time.sleep(10)

print(f'Done: {task.status()}')
```

注意点:

- `folder` は `/content/drive/MyDrive/` の直下に自動生成される
- `scale` を指定しないとデフォルトで 1度(約111km)になり莫大なデータ量になるので必ず指定する
- `maxPixels` は広範囲だと不足する場合がある。その場合は `maxPixels=1e13`などを指定

推奨フォルダ構成 (Google Drive 内)

MyDrive/

└─ astrocamp-2026/

│ └─ export/

│ │ └─ sentinel2/

│ │ └─ sentinel1/

│ └─ farmland/

│ └─ results/

└─ fallback/

API で取得するデータの実装例

GEE API (Sentinel-2 NDVI)

合宿中も回線があれば毎回 GEE から直接取得できる。以下のコードはエクスポート不要。

```
import ee
```

```
import geemap
```

```
ee.Initialize(project='ee-yourproject')
```

```
roi = ee.Geometry.Rectangle([140.5, 37.2, 141.2, 37.8])
```

```
collection = (
```

```

ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')

.filterDate('2024-06-01', '2024-06-30')

.filterBounds(roi)

.filterMetadata('CLOUDY_PIXEL_PERCENTAGE', 'less_than', 20)

)

image = collection.sort('CLOUDY_PIXEL_PERCENTAGE').first()

ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')

```

GEE API (Sentinel-1 SAR)

```

sar_collection = (

    ee.ImageCollection('COPERNICUS/S1_GRD')

    .filterDate('2024-06-01', '2024-06-30')

    .filterBounds(roi)

    .filter(ee.Filter.eq('orbitProperties_pass', 'ASCENDING'))

    .filter(ee.Filter.eq('instrumentMode', 'IW'))

)

sar_image = sar_collection.first()

vh = sar_image.select('VH')

vh_db = ee.Image.constant(10).multiply(vh.log10()).rename('VH_db')

```

JAXA Earth API (ALOS DSM・GCOM-C NDVI)

API キー不要で利用可能。詳細なコードは [tutorials/00_ai_assisted_gis.ipynb](#) Part C を参照。

```
import jaxa_earth as je
```

```

catalog = je.Catalog()

bbox = [140.5, 37.2, 141.2, 37.8]

dsm = catalog.search(collections=['ALOS PRISM AW3D30 v3.2'], bbox=bbox)

data = dsm[0].load()

ndvi_jaxa = catalog.search(

    collections=['GCOM-C SGLI NDVI v3 monthly'],

    bbox=bbox,

    datetime='2024-06'

)

```

エクスポートが必要なケースと判断基準

ケース	API で代替可能？	判断
NDVI の単発計算・可視化	✓ 可能	エクスポート不要
SAR 後方散乱の単発計算	✓ 可能	エクスポート不要
複数時期のスタックを作る	⚠ 毎回 API 呼ぶと遅い	時期数が少なければ API で十分
オフライン環境での発表・再現	✗ 通信断では使えない	エクスポート推奨 → Google Drive
他ツール(QGIS等)で読み込みたい	✗ 画像ファイルが必要	エクスポート推奨 → Google Drive
機械学習の特徴量として使う	⚠ 繰り返し読むならローカルが速い	余裕があればエクスポート

基本判断:「とりあえずエクスポート」はしない。必要な理由(オフライン対策・他ツール連携・繰り返し利用)があるときだけエクスポートする。

農地ポリゴンデータの入手

農水省 農地ポリゴンデータ

農林水産省が公開する筆ポリゴン(農地の区画データ)は [geopandas](#) で読み込んで解析に使える。

入手先:

- [農林水産省 農地データ](#) — 都道府県ごとに Shapefile または GeoJSON で公開

Colab での読み込み:

```
import geopandas as gpd
```

```
gdf =  
gpd.read_file('/content/drive/MyDrive/astrocamp-2026/export/farmland/fukushima_polygons.geojson')
```

```
print(gdf.head())
```

注意点:

- ファイルサイズは都道府県単位で数十MB 程度(福島県全体でも十分扱えるサイズ)
- 一度ダウンロードして Google Drive に保存すれば、以降は毎回ダウンロード不要
- 合宿前に最新版を確認すること

e-Stat 統計情報

相馬市・南相馬市の農業集落別耕地面積は e-Stat で確認する。

- [e-Stat](#) → 「農業」→「農林業センサス」→ 福島県 → 相馬市・南相馬市
- ブラウザ閲覧のみでダウンロードは任意

Dropbox を代替ストレージとして使う場合(任意)

Google Drive 以外のクラウドストレージを使いたい場合の手順。

```
import urllib.request
```

```
url = 'https://www.dropbox.com/s/.../file.geojson?dl=1'
```

```
urllib.request.urlretrieve(url, 'file.geojson')
```

注意点:

- Google Drive に比べて操作性は落ちる (GEE のエクスポート先にできない、マウントに一手間かかる)
- どうしても Google Drive が使えない場合の代替として検討

オフラインフォールバック計画

合宿当日の通信障害に備え、最低限これだけは確保しておく。

準備するもの (すべて Google Drive に保存)

項目	内容	サイズ目安
S-2 フォールバック用 GeoTIFF	解析対象エリアの代表的な 1 ~ 2 シーン (6月・8月など)	各数十MB
S-1 フォールバック用 GeoTIFF	同上	各数十MB
農地ポリゴン GeoJSON	福島県浜通り分	数MB
解析用 Colab ノートブック	GEE セルをスキップしてローカルファイルを読む代替セルを用意	ゼロ (コードのみ)

Colab ノートブックのオフライン対応例

```
import os
```

```
ONLINE = len(os.listdir('/content/drive/MyDrive/astrocamp-2026/fallback/')) == 0
```

```
if ONLINE:
```

```
    import ee
```

```
    ee.Initialize()
```

else:

```
import rasterio
```

```
with rasterio.open('/content/drive/MyDrive/astrocamp-2026/fallback/s2_ndvi_202406.tif') as  
src:
```

```
ndvi_array = src.read(1)
```

Week 8 チェックリスト(更新版)

合宿参加の条件。ローカル保存ではなく、**Google Drive** への保存を基本とする。

#	項目	保存先	確認
1	S-2 対象シーンを GEE からエクスポート	Google Drive(export/sentinel2/)	☐
2	S-1 対象シーンを GEE からエクスポート	Google Drive(export/sentinel1/)	☐
3	農地ポリゴンを保存	Google Drive(export/farmland/)	☐
4	Colab から Google Drive がマウントでき ることを確認	—	☐
5	ネット断でも Colab が 最後まで動作すること を確認(フォールバック 用セルの動作確認)	—	☐
6	フォールバック用 GeoTIFF を Google Drive に準備(任意だ が推奨)	Google Drive(fallback/)	☐

API で代替できる項目の確認:

- 普段の解析は GEE API / JAXA Earth API で十分 → GeoTIFF へのエクスポートはオフライン対策のみで OK
 - エクスポートする場合も「必ずローカルにダウンロード」ではなく Google Drive で管理する
-

参考リンク

- [環境構築手順](#) — ツールのアカウント登録手順
- [OpenStreetMap ガイド](#) — 水域・森林の座標を OSM から取得する方法
- [GEE チュートリアル](#) — GEE の基本操作
- [AIアシストGISチュートリアル](#) — JAXA Earth API を含むコード例
- [AIコーディングガイド](#) — プロンプトテンプレート
- [農水省 農地ポリゴンデータ](#)